

GisCampus

GIS sistem za upravljanje infrastrukturom univerzitetskog
kampusu u Novom Sadu

Luka Zelembaba i Dušan Đurić

Uvod

GisCampus je studentski projekat iz predmeta Osnove geoinformatike, namenjen prikazu i upravljanju podacima o infrastrukturi univerzitetskog kampusa u Novom Sadu. Cilj je da se podaci o zonama, zgradama, parkiralištima, zelenim površinama, infrastrukturnim objektima i sportskim terenima povežu sa njihovim položajem na interaktivnoj mapi.

Razvoj je podeljen na tri celine. SQL deo obuhvata PostgreSQL/PostGIS bazu, povezane tabele, unos i izmenu podataka i upite. GEO deo dodaje vektorske slojeve, rastersku podlogu, ručno označene objekte i prostorne analize. U ML delu primenjuje se već istreniran model za izdvajanje zgrada sa snimka. Dobijene detekcije pretvaraju se u poligone i prikazuju odvojeno od ručno evidentiranih zgrada, uz mogućnost promene statusa provere.

Dokumentacija objašnjava organizaciju projekta, izvore podataka, način rada aplikacije i modela, ostvarene rezultate i ograničenja. Projekat prikazuje odabrane objekte, a ne potpunu evidenciju kampusa. Automatske detekcije mogu sadržati greške, pa njihova pouzdanost ne zamenjuje proveru rezultata.

Sadržaj

1. Cilj projekta i zahtevi zadatka
2. Organizacija projekta i razvojni postupak
3. Baza podataka i model veza
4. Rečnik podataka
5. Python SQL, početni unos i DataFrame
6. CRUD operacije i pravila unosa
7. SQL JOIN i WHERE upiti
8. Izvori prostornih podataka i koordinatni sistemi
9. Povezivanje vektora i SQL evidencije
10. GIS interfejs i korisnički postupci
11. Overlay tehnike i prostorni upiti
12. ML model: arhitektura i poreklo
13. Obrada rastera i računanje pouzdanosti
14. Vektorizacija, dodela zona i PostGIS upis
15. Rezultati i prostorne analize ML detekcija
16. Provere, ograničenja i budući razvoj
17. Pokretanje i priprema
18. Izvori i literatura

1. Cilj projekta i zahtevi zadatka

GisCampus objedinjuje relacione podatke o infrastrukturi kampusa, njenu lokaciju i geometriju, kartografski prikaz i rezultate automatskog izdvajanja zgrada sa snimka. Područje interesa je univerzitetski kampus u Novom Sadu. Evidencija obuhvata odabrane zgrade, parkirališta, zelene površine, infrastrukturne objekte i sportske terene.

Projekat je razvijan postepeno, kroz SQL, GEO i ML deo. Najpre su napravljene tabele i uneti početni atributi. Zatim su dodate geometrije preuzete iz OSM podataka ili ručno nacrtane na mapi. Na kraju je primenjen već istreniran U-Net model za segmentaciju zgrada, čiji su rezultati pretvoreni u poligone i upisani u zasebnu tabelu.

Deo 1 - Python SQL

Zahtevi zadatka	Kako su realizovani
Kreirati PostgreSQL/PostGIS bazu i povezati se iz Python-a preko psycopg2.	Kreirana je baza <code>gis_kampus</code> , uključena PostGIS ekstenzija i napravljena konekcija pomoću podataka iz <code>.env</code> fajla. Python funkcije preko biblioteke <code>psycopg2</code> izvršavaju SQL naredbe.
Kreirati najmanje pet tabela sa 5-10 kolona, primarnim i stranim ključevima.	Napravljeno je šest osnovnih tabela: <code>zone</code> , <code>zgrade</code> , <code>parkirališta</code> , <code>zelene površine</code> , <code>infrastrukturni objekti</code> i <code>tereni</code> . Imaju po pet ili šest kolona i sopstveni primarni ključ. Sve osim roditeljske tabele <code>zona</code> imaju i strani ključ <code>zona_id</code> . Naknadno je dodata tabela <code>m1_zgrade</code> sa šest kolona.
Ručno uneti najmanje pet redova u svaku tabelu pomoću INSERT naredbi.	Za šest osnovnih tabela napisane su eksplicitne INSERT naredbe: 5 <code>zona</code> , 14 <code>zgrade</code> , 12 <code>parkirališta</code> , 5 zelenih površina, 6 infrastrukturnih objekata i 8 terena. To je ukupno 50 početnih redova. ML tabela se puni rezultatima modela, a ne ručnim početnim unosom.
Učitati sve podatke pomoću pandas biblioteke i omogućiti CRUD.	Podaci svake tabele učitavaju se u zaseban DataFrame. Implementirani su prikaz, dodavanje, izmena i brisanje redova. Aplikacija koristi ove funkcije i dodaje provere unosa i geometrije.
Napraviti 5-10 JOIN/WHERE upita i sve izvršavati kroz Python.	U <code>sql/queries.py</code> definisano je sedam upita koji povezuju tabele po stranom ključu i filtriraju podatke. Rezultati se vraćaju kao DataFrame. Tačan SQL tekst dat je u dodatku B; to su upiti nad kolonama, odvojeni od prostornih analiza na mapi.

Deo 2 - Python GEO

Zahtevi zadatka	Kako su realizovani
Preuzeti više SHP slojeva za područje u Srbiji i učitati ih pomoću GeoPandas-a.	Preuzet je Geofabrik paket za Srbiju. Za područje kampusa učitano je pet slojeva: zgrade, putevi, namena zemljišta, tačkasti i poligonski objekti. Prostorni filter ograničava učitavanje na izabrani pravougaonik.
Napraviti pandas DataFrame iz SHP podataka i spojiti ih sa SQL tabelama.	Iz učitanih GeoDataFrame objekata dobijaju se pandas DataFrame tabele uz zadržavanje geometrije. Zgrade i tereni povezani su sa izabranim OSM poligonima. Zone, parkirališta, zelenilo i infrastruktura ručno su označeni, sačuvani kao GeoJSON i povezani sa odgovarajućim redovima baze.
Omogućiti uključivanje i isključivanje slojeva, promenu simbologije i rastersku podlogu.	Na interaktivnoj mapi mogu se birati vidljivi slojevi i menjati boja, providnost i debljina ivice. Ispod vektora prikazuje se lokalno sačuvan raster sa servisa Esri World Imagery.
Napraviti najmanje pet primera overlay tehnika i prostornih upita.	Implementirano je sedam primera: clip, intersection, union, difference, buffer, within i overlaps. Rezultat izabrane analize prikazuje se kao sloj na mapi i kao tabela, bez promene izvornih podataka.

Deo 3 - Python ML

Zahtevi zadatka	Kako su realizovani
Detektovati objekte na snimcima algoritmom mašinskog učenja.	Upotrebljen je već istreniran U-Net sa ResNet34 enkoderom za izdvajanje zgrada. Model obrađuje delove rasterskog snimka i daje masku piksela koji predstavljaju zgrade.
Pretvoriti detekcije u vektore, učitati ih u PostGIS i DataFrame i prikazati na mapi.	Maska je pretvorena u poligone, mali rezultati su filtrirani, a svakom objektu dodeljena je zona najvećeg preklapanja. Rezultati se čuvaju u <code>m1_zgrade</code> , učitavaju za tabelarni prikaz i prikazuju kao zaseban sloj.
Dodati atribut i omogućiti njihovu izmenu kroz aplikaciju.	Detekcije imaju ID, zonu, površinu, pouzdanost, geometriju i status provere. Korisnik menja status provere u potvrđeno, odbijeno ili nije potvrđeno, uz istovremeno označavanje objekta na mapi.
Napraviti prostorne analize rezultata.	Dodate su dve analize: ML zgrade potpuno unutar Severne zone i presek ML detekcija sa evidentiranim zgradama. Njihova realizacija i rezultati opisani su u poglavlju 15.

2. Organizacija projekta i razvojni postupak

```
GisCampus/
  → app.py
  → requirements.txt
  → README.md

scripts/ → backup_database.ps1
         → restore_database.ps1
         → check_setup.py
         → prepare_assets.py

src/giscampus/ → config.py
               → __init__.py

src/giscampus/sql/ → __init__.py
                  → database.py
                  → crud.py
                  → queries.py

src/giscampus/geo/ → __init__.py
                  → data.py
                  → map.py
                  → analysis.py

src/giscampus/ml/ → __init__.py
                  → data.py
                  → model.py
                  → detection.py
                  → visualization.py
                  → vectorization.py

data/raw/ → vector/      (Geofabrik arhiva i SHP)
          → raster/      (rasterska podloga)
data/processed/ → campus/ (zone i ručni crteži)
data/backup/ → gis_kampus.backup (kopija sedam projektnih tabela)
data/ml/ → models/      (težine modela)
          → results/    (maske, GeoJSON i pregled)

docs/ → documentation.md
      → documentation.pdf

tests/ → test_ml_analyses.py
```

SQL - rad sa bazom

- sql/database.py uspostavlja konekciju, kreira bazu i PostGIS ekstenziju, definiše tabele i početne INSERT naredbe. Sadrži i učitavanje tabela u pandas DataFrame.
- sql/crud.py sadrži prikaz, dodavanje, izmenu i brisanje redova. Pri prostornom unosu proverava attribute i geometriju, pronalazi zonu i računa površinu.
- sql/queries.py sadrži sedam SQL upita sa JOIN i WHERE uslovima, njihove parametre i funkcije za izvršavanje. Oni povezuju podatke po ključevima; nisu clip, buffer i ostale geometrijske analize.

GEO - prostorni podaci i analize

- geo/data.py preuzima i učitava SHP podatke za izabrano područje, pravi DataFrame prikaze i priprema lokalnu rastersku podlogu.
- geo/map.py povezuje nazive evidentiranih zgrada i terena sa OSM identifikatorima, priprema njihove geometrije i pomoćne prikaze za proveru na mapi. Obuhvata i dodavanje rastera u mapu.
- geo/analysis.py priprema i upisuje geometrije zona i ostalih slojeva u PostGIS, učitava slojeve iz baze i izvršava sedam GEO analiza i dve analize ML rezultata.

ML - automatsko izdvajanje zgrada

- `ml/data.py` učitava RGB kanale rastera i priprema položaje isečaka koje model obrađuje.
- `ml/model.py` definiše U-Net sa ResNet34 enkoderom i učitava preuzete težine istreniranog modela.
- `ml/detection.py` pokreće model nad isečcima slike, objedinjuje preklopljene rezultate i čuva raster pouzdanosti i binarnu masku zgrada za područje kampusa.
- `ml/visualization.py` pravi sliku za poređenje originalnog snimka, obojenih detekcija i binarne maske.
- `ml/vectorization.py` pretvara masku u poligone, računa površinu i pouzdanost, dodeljuje zone, izvozi GeoJSON i upisuje rezultate u PostGIS.

Zajednički fajlovi

`app.py` povezuje sve delove u aplikaciju: mapu, slojeve i stilove, prikaz tabela, CRUD, proveru ML detekcija i prostorne analize. `config.py` sadrži zajednička podešavanja, uključujući podatke za konekciju. Fajlovi `__init__.py` označavaju Python pakete i ne predstavljaju zasebne korake obrade.

`scripts/backup_database.ps1` pravi prenosivu kopiju sedam projektnih tabela, a `scripts/restore_database.ps1` ih obnavlja uz obaveznu potvrdu `-Force`. `scripts/prepare_assets.py` preuzima raster i opciono težine modela, dok `scripts/check_setup.py` proverava fajlove, PostGIS i broj redova pre pokretanja.

Projekat je razvijan u VS Code-u, uz Python virtuelno okruženje `.venv` i Git/GitHub.

Komentari, nazivi domenskih atributa i objašnjenja pisani su na srpskom jeziku, dok su standardni nazivi modula i biblioteka na engleskom.

`.env` sadrži lokalne pristupne podatke i isključen je iz Git-a. Veliki rasteri, SHP arhiva, težine modela i ML rasterski izlazi ne postavljaju se u repozitorijum. Mali ručno pripremljeni GeoJSON fajlovi i rezervna kopija projektnih tabela jesu deo repozitorijuma. Zbog toga se posle kloniranja baza obnavlja iz backupa, a raster preuzima posebnom komandom.

3. Baza podataka i model veza

PostgreSQL je sistem za upravljanje relacionim bazama podataka. Projektna baza gis_kampus predstavlja zasebnu logičku celinu na PostgreSQL serveru. PostGIS proširuje PostgreSQL geometrijskim tipovima i prostornim funkcijama.

Model podataka je plan kako su podaci organizovani: koje tabele postoje, koje kolone imaju, šta predstavlja jedan red i kako su tabele povezane. U ovom projektu, na primer, jedan red u tabeli zgrade predstavlja evidentiranu zgradu, a zona_id je povezuje sa njenom zonom.

Relacioni podaci su podaci raspoređeni u tabele koje se mogu međusobno povezivati pomoću ključeva. Naziv zone zato ne moramo ponavljati u svakom redu zgrada: čuvamo njen ID, a JOIN upitom dobijamo naziv i ostale podatke iz tabele zona. Baza nije isto što i tabela - projektna baza sadrži sve ove povezane tabele.

Tabela zone_kampusu sadrži Severnu, Južnu, Istočnu, Zapadnu i Centralnu zonu, sa oznakama S, J, I, Z i C. Ona je roditeljska tabela. Sve ostale tabele imaju zona_id koji upućuje na zone_kampusu.zona_id. Jedna zona može sadržati više drugih objekata. Sama zona nema dodat strani ključ jer u ovom modelu ne zavisi od druge tabele.

Tabela	Redova	Geometrija
zone_kampusu	5	Polygon
zgrade	14	MultiPolygon
parkiralista	12	Polygon
zelene_povrsine	5	Polygon
infrastrukturni_objekti	6	Point
tereni	8	Polygon
ml_zgrade	56	Polygon

Primarni ključevi koriste INTEGER GENERATED ALWAYS AS IDENTITY. ID nije redni broj reda u prikazu i ne mora ostati neprekinut posle brisanja. Nazivi infrastrukturnih objekata ne moraju biti jedinstveni; dva reda mogu imati isti naziv i razlikovati se po ID-u. ON DELETE RESTRICT sprečava brisanje zone dok postoje redovi koji je koriste.

4. Rečnik podataka

Tabela	Kolone i osnovni tipovi
zone_kampus_a	zona_id INTEGER (PK); naziv VARCHAR(50), UNIQUE; oznaka VARCHAR(5), UNIQUE; površina_m2 NUMERIC(12,2); geometrija Polygon.
zgrade	zgrada_id INTEGER (PK); zona_id INTEGER (FK); naziv VARCHAR(120); tip VARCHAR(60); površina_m2 NUMERIC(12,2); geometrija MultiPolygon.
parkiralista	parkiraliste_id INTEGER (PK); zona_id INTEGER (FK); naziv VARCHAR(50), UNIQUE; tip VARCHAR(20); površina_m2 NUMERIC(12,2); geometrija Polygon.
zelene_povrsine	zelena_povrsina_id INTEGER (PK); zona_id INTEGER (FK); tip VARCHAR(60); površina_m2 NUMERIC(12,2); geometrija Polygon.
infrastrukturni_objekti	infrastrukturni_objekat_id INTEGER (PK); zona_id INTEGER (FK); naziv VARCHAR(120); stanje VARCHAR(30); geometrija Point.
tereni	teren_id INTEGER (PK); zona_id INTEGER (FK); naziv VARCHAR(120); površina_m2 NUMERIC(12,2); geometrija Polygon.
ml_zgrade	ml_zgrada_id INTEGER (PK); zona_id INTEGER (FK); površina_m2 NUMERIC(12,2); pouzdanost NUMERIC(5,4); status_provere VARCHAR(30); geometrija Polygon.

Sve geometrije u bazi definisane su u SRID 32634 koordinatnom sistemu. UNIQUE znači da se ista vrednost ne sme ponoviti u toj koloni.

Kolone geometrija i površina_m2 postoje od kreiranja osnovnih tabela, ali pri početnom ručnom unosu imaju vrednost NULL, jer objekti tada još nisu povezani sa prostornim podacima. Njihove geometrije i površine popunjavaju se kasnije, u GEO delu. NULL ovde znači da podatak još nije unet, a ne da kolona ne postoji ili da baza nije povezana sa PostGIS-om.

ML tabela zahteva popunjenu geometriju, površinu i pouzdanost. Pouzdanost ima CHECK ograničenje od 0 do 1, a status dozvoljava nije_potvrđeno, potvrđeno ili odbijeno. Početna vrednost statusa je nije_potvrđeno.

5. Python SQL, početni unos i DataFrame

Povezivanje je realizovano bibliotekom `psycopg2`. Funkcija `ucitaj_podesavanja_baze()` čita `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER` i `DB_PASSWORD` iz lokalnog `.env` fajla.

`pripremi_bazu()` redom poziva `kreiraj_bazu()`, `ukljuci_postgis()`, `kreiraj_tabele()` i `unesi_pocetne_podatke()`. Kreiranje tabela izvršava se pozivom `cursor.execute(SQL_KREIRANJE_TABELA)`, odnosno SQL naredbe `CREATE TABLE` pokreće Python biblioteka.

Početni unos je napisan eksplicitnim SQL `INSERT` naredbama u `SQL_UNOS_POADATAKA`. Zone se unose prve, a drugi redovi koriste podupit po oznaci zone da dobiju stvarni `zona_id`.

Sadržaj početne evidencije

Zgrade obuhvataju Tehnološki, Poljoprivredni, Pravni, Filozofski, Fakultet tehničkih nauka, Prirodno-matematički i Ekonomski fakultet; Visoku poslovnu školu; Naučno-tehnološki park; domove Slobodan Bajić i Veljko Vlahović; Veseli vrtić; Rektorat i Institut BioSens. Ekonomski fakultet vodi se u Zapadnoj, a FTN u Centralnoj zoni.

Parkirališta imaju nazive `parkiraliste_1` do `parkiraliste_12`: šest javnih i šest privatnih. Zelene površine su park u Istočnoj, livada u Severnoj i tri dvorišta u Zapadnoj zoni. Infrastrukturu čine dve trafostanice, fontana, dva parkirališta za bicikle i kontejner. Fontana je neispravna. Osam terena je evidentirano u Južnoj zoni: fudbal, tri mala fudbala, dve košarke, tenis i odbojka.

6. CRUD operacije i pravila unosa

Operacija	Funkcija i ponašanje
Create	<code>dodaj_red()</code> upisuje prosleđene vrednosti u tabelu. <code>dodaj_prostorni_red()</code> služi za unos iz aplikacije i pre upisa dodatno proverava attribute i nacrtanu geometriju, određuje zonu i računa površinu.
Read	<code>prikazi_sve()</code> vraća redove izabrane tabele kao DataFrame.
Update	<code>azuriraj_red()</code> menja dozvoljene attribute odabranog ID-a.
Delete	<code>obrisi_red()</code> briše red po primarnom ključu.

Za dodavanje kroz aplikaciju popunjavaju se svi traženi atributi. Prazan tekst i tekst sastavljen samo od razmaka nisu dovoljni. Geometrija mora biti nacrtana, neprazna, ispravna i odgovarajućeg tipa. Infrastrukturni objekti crtaju se kao tačke, a ostali kao poligoni. Za zgrade se poligon pri upisu pretvara u MultiPolygon.

Koordinate crteža iz mape transformišu se iz EPSG:4326 u EPSG:32634. Površina se automatski računa iz geometrije u metrima i zaokružuje na dve decimale.

Automatska dodela zone pri novom unosu

`pronadji_zonu_za_geometriju()` traži zonu za koju `ST_Covers(zona, objekat)` vraća tačno. Ceo objekat mora biti unutar zone ili na njenoj granici; nijedan deo ne sme biti van nje. Ovo pravilo se ne primenjuje na samu novu zonu. [1]

Pri početnim INSERT naredbama zone su birane ručno. Pri novom unosu kroz aplikaciju strani ključ računa se prostorno. Kod ML rezultata važi drugačije pravilo - zona najvećeg preklapanja, opisano u poglavlju 14.

ML tabela u korisničkom interfejsu nema opcije za ručno dodavanje ili brisanje. Dozvoljena je samo promena statusa provere.

7. SQL JOIN i WHERE upiti

Spajanje dve ili više tabela preko stranog ključa i filtriranje podataka pomoću WHERE uslova. Implementirano u `src/giscampus/sql/queries.py`.

Upit	Povezane tabele i rezultat
<code>zgrade_u_centralnoj_zoni</code>	Spaja zgrade i zone. Filtrira oznaku zone C i prikazuje ID, naziv i tip zgrade, uz naziv njene zone.
<code>fakulteti_u_kampusu</code>	Spaja zgrade i zone. Izdvaja zgrade čiji je tip fakultet i prikazuje njihove nazive i zone.
<code>javna_parkiralista</code>	Spaja parkirališta i zone. Izdvaja tip javno i prikazuje ID parkirališta, tip i naziv zone.
<code>zelene_povrsine_u_istocnoj_zoni</code>	Spaja zelene površine i zone. Filtrira oznaku zone I i prikazuje ID i tip zelene površine, uz naziv zone.
<code>neispravni_infrastrukturni_objekt_i</code>	Spaja infrastrukturne objekte i zone. Izdvaja stanje neispravno i prikazuje ID, naziv i stanje objekta, uz njegovu zonu.
<code>tereni_u_juznoj_zoni</code>	Spaja terene i zone. Filtrira oznaku zone J i prikazuje ID i naziv terena, uz naziv zone.
<code>zgrade_i_javna_parkiralista_po_zonama</code>	Spaja tri tabele: zone, zgrade i parkirališta. Izdvaja javna parkirališta i za svaku prikazanu zonu računa broj različitih zgrada i javnih parkirališta.

8. Izvori podataka i koordinatni sistemi

Vektorski podaci preuzeti su iz Geofabrik SHP paketa za Srbiju zasnovanog na OpenStreetMap podacima. Arhiva je preuzeta u data/raw/vector/. [2]

Ceo paket Srbije je prvobitno preuzet, a zatim učitani pravougaoni obuhvat (19.8470, 45.2425, 19.8590, 45.2500),.

Sloj	SHP naziv	Učitanih objekata
Zgrade	gis_osm_buildings_a_free_1.shp	176
Putevi	gis_osm_roads_free_1.shp	726
Namena zemljišta	gis_osm_landuse_a_free_1.shp	158
Tačkasti objekti	gis_osm_pois_free_1.shp	226
Poligonski objekti	gis_osm_pois_a_free_1.shp	42

Ovi brojevi odnose se na lokalno korišćenu verziju SHP podataka, ne na celokupnu Srbiju.

Rasterska podloga preuzeta je sa Esri World Imagery export servisa i sačuvana je kao data/raw/raster/kampus_esri.tif. Servisu se prosleđuju bboxSR=4326, imageSR=4326 i dimenzije 2400 x 1500. Sačuvani TIFF ima četiri kanala, dok prikaz i model koriste prva tri kao RGB. [3]

EPSG:4326 koristi geografske koordinate u stepenima i služi za razmenu sa veb mapom.

9. Povezivanje vektora i SQL evidencije

Geometrije se čuvaju u .shp, atributi u .dbf, indeks u .shx, opis koordinatnog sistema u .prj. Ove datoteke zajedno čine sloj. GeoJSON takođe čuva geometrije i attribute u jednoj tekstualnoj strukturi.

Za zgrade je napravljen Python rečnik OSM_POLIGONI_ZGRADA u geo/map.py. Ključevi su nazivi zgrada iz SQL evidencije, a vrednosti jedan ili više osm_id brojeva iz SHP-a.

FTN je povezan sa šest izvornih poligona, PMF sa sedam, a druge evidentirane zgrade sa jednim. Geometrije se spajaju i predstavljaju kao MultiPolygon. Za Rektorat je odabran samo centralni objekat. Ukupno je povezano 14 evidentiranih zgrada, što ne znači da na kampusu zaista postoji samo 14 zgrada.

Ručno crtani slojevi

Nacrtni je okvir kampusa i pet zona. `ocisti_zone()` deli linijsku mrežu granica na poligonske delove, odseca ih okvirom i dodeljuje zonama prema preklapanju. Cilj je uklanjanje rupa i preklapanja u početnoj podeli. Konačne zone čuvaju se u `data/processed/campus/zone.geojson`.

Parkirališta i zelene površine nacrtani su ručno preko rastera, a infrastrukturni objekti označeni tačkama. Ovi slojevi povezani su sa postojećim SQL redovima preko primarnih ključeva i dodatno naziva ili tipa. Ručni crteži čuvaju se kao `parkirališta.geojson`, `zelene_povrsine.geojson` i `infrastrukturni_objekti.geojson`. Pri upisu su izračunate površine poligona u EPSG:32634.

Svih 8 tereni preuzeti su iz poligonskih OSM objekata.

Napomena: GeoJSON datoteke i baza nisu automatski sinhronizovana dvosmerna kopija: kasnije izmene baze mogu zahtevati osvežavanje izvoznih fajlova.

10. GIS interfejs i korisnički postupci

Aplikacija se pokreće komandom `python -m streamlit run app.py`. Streamlit obezbeđuje stranicu, bočne kontrole i tabelarni prikaz. Folium generiše Leaflet mapu, a streamlit-folium prenosi informacije o interakciji iz mape u Python. Podloga je lokalni raster prikazan preko ImageOverlay, a projektni vektori učitavaju se iz PostGIS baze.

Na mapi se zasebno uključuje i isključuje prikaz svih slojeva. Kontrola stila omogućava promenu boje, providnosti i debljine ivice.

Bočna traka omogućava izbor tabele i CRUD operacije.

11. Overlay tehnike i prostorni upiti

Overlay operacije kombinuju geometrije slojeva i stvaraju prostorni rezultat sa novim ili izmenjenim granicama. Prostorni upiti proveravaju odnos između objekata i biraju one koji ispunjavaju uslov. [4]

Analize u `geo/analysis.py` učitavaju potrebne podatke iz PostGIS-a kao `GeoDataFrame` u EPSG:32634. Rezultat se u aplikaciji transformiše za mapu, prikazuje kao poseban sloj i kao tabela ispod mape. Analize ne menjaju izvorne redove.

Primer	Značenje i trenutni rezultat
Clip - zgrade u Severnoj zoni	Iseca zgrade granicom zone; zadržava samo njihove delove unutar nje.
Intersection - parking i zelenilo	Vraća zajedničke delove dva sloja.
Union - zelene površine i parkinzi	Zadržava obe površine i njihove attribute; preklapanja, kada postoje, izdvajaju se.
Difference - kampus bez zgrada	Od objedinjenih zona oduzima evidentirane zgrade.
Buffer - 30 m oko infrastrukture	Pravi okolinu od 30 metara oko svake tačke.
Within - infrastruktura u Centralnoj zoni	Izdvađa tačke unutar centralnog poligona.
Overlaps - parkinzi i zgrade	Traži delimično preklapanje površina, bez potpunog sadržavanja.

12. Arhitektura i poreklo ML modela

Za automatsko izdvajanje zgrada korišćen je već istreniran U-Net sa ResNet34 enkoderom. Težine `unet_bldg_instance.pth` preuzete su iz repozitorijuma `nilsho01/unet-resnet34-vhr-buildings` na Hugging Face-u. Autor modela navodi treniranje na skupu `hotosm/vhr-building-segmentation`, sa OpenAerialMap snimcima i OSM oznakama. To je spoljni model, nije treniran na ovom projektu. [5], [6]

U-Net je konvoluciona mreža za segmentaciju: izlaz daje ocenu za svaki piksel, umesto samo jedne klase za celu sliku. Enkoder smanjuje prostorne dimenzije i izdvaja karakteristike, a dekodeer povećava rezoluciju rezultata. Preskočne veze prenose prostorne detalje iz enkodera ka odgovarajućim nivoima dekodera. ResNet34 koristi rezidualne veze za prenos informacija kroz dublje blokove. [7], [8]

Podešavanje modela	Vrednost
Biblioteka	<code>segmentation-models-pytorch</code> , <code>smp.Unet</code>
Enkoder / dubina	<code>resnet34</code> / 5 nivoa
Ulaz	3 kanala, RGB; isečak 256 x 256
Kanali dekodera	256, 128, 64, 32, 16
Normalizacija dekodera	Batch normalization
Attention / interpolacija	Bez dodatnog attention modula / nearest
Izlaz	3 kanala; <code>activation=None</code>
Dodatna klasifikaciona glava	Nije uključena: <code>aux_params=None</code>
Izvršavanje	CPU, <code>model.eval()</code> , <code>torch.inference_mode()</code>

Model koristi jedan izlazni kanal - masku zgrada koja govori o sigurnosti modela u to da li piksel pripada zgradi: `torch.sigmoid(izlaz[:, 0])`.

Spoljašnje treniranje

Prema dokumentaciji preuzetog modela, za njegovo originalno treniranje korišćeni su: AdamW, batch 32, learning rate $1e-4$ za enkoder i $1e-3$ za dekodeer, weight decay $1e-4$ i zaustavljanje na 23. epohi od najviše 50. To nisu parametri našeg lokalnog treniranja, jer ga nismo sproveli. [5]

13. Obrada rastera i pouzdanost

ml/data.py učitava prva tri kanala rastera i čuva njegov profil. Snimak se deli na isečke 256 x 256, sa preklapanjem 64 piksela. Korak je zato 192 piksela. Lista početnih pozicija uključuje i poslednji isečak koji pokriva kraj slike. Po četiri isečka obrađuju se u jednom paketu.

Ulazni raspored je broj primera x broj kanala x visina x širina. RGB vrednosti pretvaraju se u float i dele sa 255, pa su u opsegu 0-1. Ne primenjuje se dodatna standardizacija ImageNet srednjim vrednostima.

Izlaz prvog kanala pretvara se sigmoid funkcijom u ocenu između 0 i 1. Za piksel koji je obrađen u više preklapljenih isečaka koristi se prosek svih ocena.

```
p = sigmoid(izlaz_prvog_kanala)
p_konacno = zbir_ocena_piksela / broj_predvidjanja
maska = 1 ako je p_konacno >= 0.5, inače 0
```

Maska zona određuje obuhvat kampusa. Pikseli van svih zona dobijaju vrednost nula pre formiranja binarne maske. To ograničava rezultate na kampus, ali može odseći deo objekta koji izlazi izvan spoljnog obuhvata kampusa.

Rezultati se čuvaju u data/ml/results/: verovatnoca_zgrada.tif sadrži float32 ocene, maska_zgrada.tif binarne uint8 vrednosti, a pregled_detekcije.png original, detekciju prikazu preko originalnog snimka i binarnu masku. GeoTIFF izlazi zadržavaju georeferenciranje originalnog rastera.

Pouzdanost jednog detektovanog objekta

_prosecna_pouzdanost() u vectorization.py izdvaja vrednosti rastera verovatnoće unutar poligona i računa njihovu aritmetičku sredinu. Na primer, za ocene 0.90, 0.80, 0.95 i 0.85 prosek je 0.875, odnosno 87.5%. Računanje se vrši nad poligonom nastalim iz maske pre pojednostavljivanja njegovih granica. Vrednost se za bazu zaokružuje na četiri decimale.

Pouzdanost nije izmerena tačnost modela niti kalibrisana verovatnoća da je ceo objekat zaista zgrada. Model može biti samouveren i kada greši. Prosek se računa nad već izdvojenim pikselima detekcije. Zato se odvojeno čuva status ljudske provere.

14. Vektorizacija, dodela zone i PostGIS upis

`vektORIZUJ_detekcije()` čita masku i raster verovatnoće. `rasterio.features.shapes()` pretvara povezane piksele vrednosti 1 u poligonske geometrije. Transformacija rastera obezbeđuje stvarne koordinate, a `shapely.geometry.shape()` pretvara dobijeni opis u geometrijski objekat. Rezultati se stavljaju u `GeoDataFrame`. [9]

Poligoni se transformišu u EPSG:32634. `explode()` razdvaja višedelne geometrije, a `simplify(tolerance=0.35, preserve_topology=True)` pojednostavljuje granice. Površina se računa posle pojednostavljivanja. Objekti manji od 20 m² uklanjaju se kao kandidati za šum. Time se mogu ukloniti i stvarni mali objekti, pa prag predstavlja praktičnu pretpostavku, ne univerzalno pravilo.

Dodela zone najvećeg preklapanja

Učita se `zone.geojson` i izračuna `intersection` svake detekcije sa zonama. Površine tih privremenih preseka sortiraju se opadajuće. Za svaku detekciju zadržava se naziv zone sa najvećim presekom. Konačni rezultat uzima originalni poligon detekcije, a ne isečeni poligon preseka. Tako jedna zgrada ostaje jedan objekat i kada prelazi unutrašnju granicu zona.

Izraz većinski deo ovde znači najveći među izračunatim presecima; kod ne zahteva da pobednički deo obavezno bude veći od 50% ukupne površine.

`upisi_u_postgis()` čita nazive i ID-eve zona iz baze i formira rečnik naziv -> zona_id. Time se tekstualna zona iz vektorizacije pretvara u strani ključ.

Dve odvojene evidencije zgrada

`ml_zgrade` ne sadrži strani ključ ka zgrade. Evidentirane zgrade i automatski rezultati prikazuju se kao odvojeni slojevi. Ne pretpostavlja se jedan-na-jedan odnos zbog snimka relativno male rezolucije.

Rezultat se izvozi i u `ml_zgrade.geojson` u EPSG:4326. Njegov `detekcija_id` nije isti koncept kao baza `ml_zgrada_id` i ne treba se oslanjati na njihovu trajnu jednakost. Funkcija podrazumevano preskače novi upis ako ML tabela već ima redove. Eksplicitna zamena postojećih rezultata odbija se ako je neki status već promenjen. GeoJSON iz prethodnog izvoza ne osvežava se automatski kada korisnik promeni status u bazi.

15. Rezultati i ML prostorne analize

Pokazatelj, presek 31.08.2026.	Vrednost
ML poligona u bazi	56
Nepotvrđeno / potvrđeno / odbijeno	48 / 4 / 4
Najmanja / prosečna / najveća pouzdanost	0.5475 / 0.8830 / 0.9924
Najmanja / prosečna / najveća atributska površina	31.72 / 732.10 / 5053.02 m2
Zbir površina ML geometrija	Približno 40997.36 m2
Neispravne geometrije u proveru svih tabela	0

Zona po stranom ključu	Broj ML detekcija
Severna	10
Južna	19
Istočna	2
Zapadna	15
Centralna	10

Zbir atributskih površina zaokruženih po redovima može se malo razlikovati od površine dobijene iz punih geometrija. Ukupan broj detekcija uključuje i nepotvrđene i odbijene rezultate.

Nepoklapanje detekcija sa evidentiranim zgradama nije automatski greška modela jer evidencija nije kompletna. Ni samo poklapanje ne dokazuje da je cela detekcija tačna. Ovi rezultati služe za prostorno istraživanje i pomoć pri vizuelnoj proveru, ne zamenjuju nezavisno označen test skup.

Vizuelni izlaz detekcije



Slika 1. Postojeći rezultat lokalne detekcije: originalni raster (levo), cijan označene detekcije (sredina) i binarna maska (desno). Podloga: Esri World Imagery i dobavljači snimaka. Ovo nije snimak ekrana korisničkog interfejsa.

Vizuelni pregled omogućava da se proceni da li obojene površine prate krovove, da li se susedne zgrade spajaju i da li se javljaju lažne detekcije na drugim površinama. Binarna maska pokazuje piksele pre konačnog filtriranja poligona po površini i ne treba je poistovetiti sa tačnim konačnim vektorskim slojem iz baze.

Snimak ima ograničenu rezoluciju i ne prikazuje sve detalje jednako jasno. Senke, drveće, boja krova, ortorektifikacija i razlike datuma snimaka mogu uticati na poklapanje. Korisnik zato u aplikaciji odvojeno uključuje raster, evidentirane zgrade i ML sloj, bira detekciju i menja njen status nakon pregleda.

Za prihvatanje rezultata na odbrani treba pokazati makar jedan primer dobro izdvojenog krova, jedan granični slučaj i jedan odbijen rezultat. Ne treba birati samo najbolje primere i predstavljati ih kao dokaz ukupne tačnosti.

16. Provere i ograničenja

Provere

U proverenom preseku svih sedam tabela ukupno je 106 redova: 50 osnovnih i 56 ML redova. Svi redovi imaju geometriju i u proveru ST_IsValid nije pronađena neispravna geometrija.

Ograničenja trenutne verzije

Nema kompletnog nezavisnog referentnog skupa za kampus, pa nisu izmereni lokalni precision, recall, F1 ili IoU (Intersection over Union). Evidentirane zgrade su delom zasnovane na OSM-u, dok je i trening spoljnog modela koristio OSM oznake; zato ovo poređenje nije automatski nezavisna validacija.

Prag 0.5 i minimalna površina 20 m2 nisu optimizovani na lokalnom validacionom skupu. Podloga i OSM evidencija mogu poticati iz različitih perioda.

Verzije direktnih Python biblioteka zaključane su u requirements.txt prema proverenom lokalnom okruženju. Geofabrik latest adresa i model main adresa ipak mogu kasnije vratiti novije podatke ili težine. Prenos projektnih tabela i ručnih crteža sada je podržan backup/restore skriptama, ali backup mora ponovo da se napravi nakon važnih izmena baze. Aplikacija nije testirana kao javni servis sa više korisnika.

17. Pokretanje i priprema za odbranu

Postavljanje na novom računaru

```
# Python 3.12 okruženje i zaključane biblioteke
py -3.12 -m venv .env
.\.env\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt

# Lokalna podešavanja; zatim ručno upisati PostgreSQL lozinku u .env
Copy-Item .env.example .env
```

Na računaru moraju biti instalirani PostgreSQL i PostGIS. Projekat je razvijen sa Python 3.12, PostgreSQL 17.11 i PostGIS 3.6.2. .env se pravi lokalno jer sadrži lozinku i ne ulazi u Git.

Baza, raster i provera

```
# Obnavlja svih sedam tabela, geometrije i trenutne ML statuse
.\scripts\restore_database.ps1 -Force

# Preuzima rastersku podlogu koja zbog veličine nije u Git-u
python scripts/prepare_assets.py

# Proverava .env, fajlove, PostGIS i sve projektne tabele
python scripts/check_setup.py

# Pokretanje aplikacije
python -m streamlit run app.py
```

Rezervna kopija data/backup/gis_kampus.backup sadrži šemu i podatke sedam projektnih tabela, uključujući geometrije, strane ključeve, identity brojače i statuse ML provere. Restore skripta zamenjuje te tabele i zato zahteva eksplicitni parametar -Force; odbija systemske baze. Raster se zasebno preuzima sa Esri servisa. Potpuno uputstvo i redosled koraka dati su u README.md.

Nakon promene podataka ili statusa treba osvežiti rezervnu kopiju:

```
.\scripts\backup_database.ps1 -Force
```

ML obrada i testovi

```
# Ponovno pokretanje modela prepisuje izlazne rastere
python scripts/prepare_assets.py --with-model
python -m src.giscampus.ml.detection
# Vektorizacija; postojeći ML redovi se podrazumevano ne zamenjuju
python -m src.giscampus.ml.vectorization
# Provera prostornih analiza bez izmene baze
python -m unittest discover -s tests -p test_ml_analyses.py -v
```

Za običan pregled aplikacije nije potrebno ponovo pokretati ML model. On se izvršava zasebno, a aplikacija čita sačuvane rezultate iz baze. Ponovno generisanje GeoJSON-a i preskakanje postojećeg SQL upisa može ostaviti različite verzije datoteke i baze, pa te korake treba pokretati svesno.

Predlog demonstracije: prikazati strukturu tabela i FK, izvršiti jedan JOIN upit, pokazati raster i vektore, objasniti EPSG, demonstrirati uključivanje slojeva i izbor objekta, dodati test objekat validnom geometrijom, pokazati jednu GEO analizu, zatim ML sloj i status provere, i na kraju dve ML analize. Test podatke ukloniti samo uz proveru tačnog ID-a.

Pre odbrane treba pokrenuti python scripts/check_setup.py, proveriti aplikaciju i napraviti svež backup. Biblioteke za veb mapu mogu i dalje koristiti spoljne JavaScript/CSS resurse, pa potpuna offline funkcionalnost nije dokazana samo time što je raster lokalni.

18. Izvori i literatura

Izvori su numerisani redom prvog pojavljivanja u tekstu. Ponovljeni broj označava ponovni poziv na isti izvor, a ne novu referencu.

[1] PostGIS - ST_Covers

https://postgis.net/docs/ST_Covers.html

Definicija prostornog predikata korišćenog pri dodavanju objekata.

[2] Geofabrik - OpenStreetMap podaci za Srbiju

<https://download.geofabrik.de/europe/serbia.html>

Izvor vektorskih SHP slojeva.

[3] Esri World Imagery - ArcGIS servis

https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer

Izvor projektne rasterske podloge.

[4] GeoPandas - Set operations with overlay

https://geopandas.org/en/stable/docs/user_guide/set_operations.html

Dokumentacija za overlay operacije nad GeoDataFrame objektima.

[5] nilsho01/unet-resnet34-vhr-buildings

<https://huggingface.co/nilsho01/unet-resnet34-vhr-buildings>

Istrenirani model i težine unet_bldg_instance.pth.

[6] HOTOSM - VHR building segmentation

<https://huggingface.co/datasets/hotosm/vhr-building-segmentation>

Skup koji autor modela navodi kao izvor treniranja; nije lokalni označeni skup za Novi Sad.

[7] Ronneberger, Fischer, Brox (2015): U-Net

<https://arxiv.org/abs/1505.04597>

Osnovni rad o U-Net arhitekturi.

[8] He i saradnici (2015): Deep Residual Learning

<https://arxiv.org/abs/1512.03385>

Osnovni rad o rezidualnim neuronskim mrežama.

[9] Rasterio - Vector Features

<https://rasterio.readthedocs.io/en/stable/topics/features.html>

Ekstrakcija vektorskih geometrija iz rastera.

Interni izvori: app.py; src/giscampus/sql/database.py, crud.py i queries.py; src/giscampus/geo/data.py, map.py i analysis.py; src/giscampus/ml/model.py, data.py, detection.py, vectorization.py i visualization.py; tests/test_ml_analyses.py; lokalni PostGIS podaci i sačuvani rezultati detekcije.

Licencne napomene nisu pravno tumačenje. Pre javnog objavljivanja proveriti konkretne uslove za kod, težine, snimke i izvedene podatke. Model i snimci nisu predstavljeni kao autorski materijal studentskog tima.

Dodatak A. SQL definicije tabela

Sledeći tekst je preuzet iz trenutne konstante SQL_KREIRANJE_TABELA. Pokazuje konkretne tipove, ograničenja i veze. Izostavljeno je izvršavanje: dokumentacija ne menja bazu.

```
-- Prostorne zone kampusa Univerziteta u Novom Sadu.
CREATE TABLE IF NOT EXISTS zone_kampusa (
    zona_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    naziv VARCHAR(50) NOT NULL UNIQUE,
    oznaka VARCHAR(5) NOT NULL UNIQUE,
    površina_m2 NUMERIC(12, 2) CHECK (površina_m2 > 0),
    geometrija geometry(Polygon, 32634)
);
-- Poznate zgrade koje se ručno evidentiraju u sistemu.
CREATE TABLE IF NOT EXISTS zgrade (
    zgrada_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    zona_id INTEGER NOT NULL REFERENCES zone_kampusa(zona_id) ON DELETE RESTRICT,
    naziv VARCHAR(120) NOT NULL,
    tip VARCHAR(60) NOT NULL,
    površina_m2 NUMERIC(12, 2) CHECK (površina_m2 > 0),
    geometrija geometry(MultiPolygon, 32634)
);
-- Parking površine koje se posmatraju kao celine.
CREATE TABLE IF NOT EXISTS parkiralista (
    parkiraliste_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    zona_id INTEGER NOT NULL REFERENCES zone_kampusa(zona_id) ON DELETE RESTRICT,
    naziv VARCHAR(50) NOT NULL UNIQUE,
    tip VARCHAR(20) NOT NULL CHECK (tip IN ('javno', 'privatno')),
    površina_m2 NUMERIC(12, 2) CHECK (površina_m2 > 0),
    geometrija geometry(Polygon, 32634)
);
-- Parkovi, livade, travnjaci i druge zelene površine.
CREATE TABLE IF NOT EXISTS zelene_povrsine (
    zelena_povrsina_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    zona_id INTEGER NOT NULL REFERENCES zone_kampusa(zona_id) ON DELETE RESTRICT,
    tip VARCHAR(60) NOT NULL,
    površina_m2 NUMERIC(12, 2) CHECK (površina_m2 > 0),
    geometrija geometry(Polygon, 32634)
);
-- Odabrani infrastrukturni objekti predstavljeni tačkama.
CREATE TABLE IF NOT EXISTS infrastrukturni_objekti (
    infrastrukturni_objekat_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    zona_id INTEGER NOT NULL REFERENCES zone_kampusa(zona_id) ON DELETE RESTRICT,
    naziv VARCHAR(120) NOT NULL,
    stanje VARCHAR(30) NOT NULL,
    geometrija geometry(Point, 32634)
);
-- Sportski tereni u južnom delu kampusa.
CREATE TABLE IF NOT EXISTS tereni (
    teren_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    zona_id INTEGER NOT NULL REFERENCES zone_kampusa(zona_id) ON DELETE RESTRICT,
    naziv VARCHAR(120) NOT NULL,
    površina_m2 NUMERIC(12, 2) CHECK (površina_m2 > 0),
    geometrija geometry(Polygon, 32634)
);
-- Zgrade automatski izdvojene iz ortofoto snimka ML modelom.
CREATE TABLE IF NOT EXISTS ml_zgrade (
    ml_zgrada_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    zona_id INTEGER NOT NULL REFERENCES zone_kampusa(zona_id) ON DELETE RESTRICT,
    površina_m2 NUMERIC(12, 2) NOT NULL CHECK (površina_m2 > 0),
    pouzdanost NUMERIC(5, 4) NOT NULL CHECK (pouzdanost BETWEEN 0 AND 1),
    status_provere VARCHAR(30) NOT NULL DEFAULT 'nije_potvrđeno'
    CHECK (status_provere IN ('nije_potvrđeno', 'potvrđeno', 'odbijeno')),
    geometrija geometry(Polygon, 32634) NOT NULL
);
```


Dodatak B. Tačan tekst SQL upita

Upiti su preuzeti iz konstante SQL_UPITI u sql/queries.py. Parametri su prikazani odvojeno, kao pri izvršavanju preko SQLAlchemy.

zgrade_u_centralnoj_zoni

```
SELECT
    z.zgrada_id,
    z.naziv AS zgrada,
    z.tip,
    zk.naziv AS zona
FROM zgrade AS z
JOIN zone_kampusu AS zk ON z.zona_id = zk.zona_id
WHERE zk.oznaka = :oznaka
ORDER BY z.naziv;
```

Parametri: {'oznaka': 'C'}

fakulteti_u_kampusu

```
SELECT
    z.zgrada_id,
    z.naziv AS fakultet,
    zk.naziv AS zona
FROM zgrade AS z
JOIN zone_kampusu AS zk ON z.zona_id = zk.zona_id
WHERE z.tip = :tip
ORDER BY zk.naziv, z.naziv;
```

Parametri: {'tip': 'fakultet'}

javna_parkiralista

```
SELECT
    p.parkiraliste_id,
    p.tip,
    zk.naziv AS zona
FROM parkiralista AS p
JOIN zone_kampusu AS zk ON p.zona_id = zk.zona_id
WHERE p.tip = :tip
ORDER BY zk.naziv, p.parkiraliste_id;
```

Parametri: {'tip': 'javno'}

zelene_povrsine_u_istocnoj_zoni

```
SELECT
    zp.zelena_povrsina_id,
    zp.tip,
    zk.naziv AS zona
FROM zelene_povrsine AS zp
JOIN zone_kampusu AS zk ON zp.zona_id = zk.zona_id
WHERE zk.oznaka = :oznaka
ORDER BY zp.zelena_povrsina_id;
```

Parametri: {'oznaka': 'I'}

neispravni_infrastrukturni_objekti

```
SELECT
    io.infrastrukturni_objekat_id,
    io.naziv AS objekat,
    io.stanje,
    zk.naziv AS zona
FROM infrastrukturni_objekti AS io
JOIN zone_kampus_a AS zk ON io.zona_id = zk.zona_id
WHERE io.stanje = :stanje
ORDER BY io.naziv;
```

Parametri: {'stanje': 'neispravno'}

tereni_u_juznoj_zoni

```
SELECT
    t.teren_id,
    t.naziv AS teren,
    zk.naziv AS zona
FROM tereni AS t
JOIN zone_kampus_a AS zk ON t.zona_id = zk.zona_id
WHERE zk.oznaka = :oznaka
ORDER BY t.teren_id;
```

Parametri: {'oznaka': 'J'}

zgrade_i_javna_parkiralista_po_zonama

```
SELECT
    zk.naziv AS zona,
    COUNT(DISTINCT z.zgrada_id) AS broj_zgrada,
    COUNT(DISTINCT p.parkiraliste_id) AS broj_javnih_parkiralista
FROM zone_kampus_a AS zk
JOIN zgrade AS z ON zk.zona_id = z.zona_id
JOIN parkiralista AS p ON zk.zona_id = p.zona_id
WHERE p.tip = :tip
GROUP BY zk.zona_id, zk.naziv
ORDER BY zk.naziv;
```

Parametri: {'tip': 'javno'}